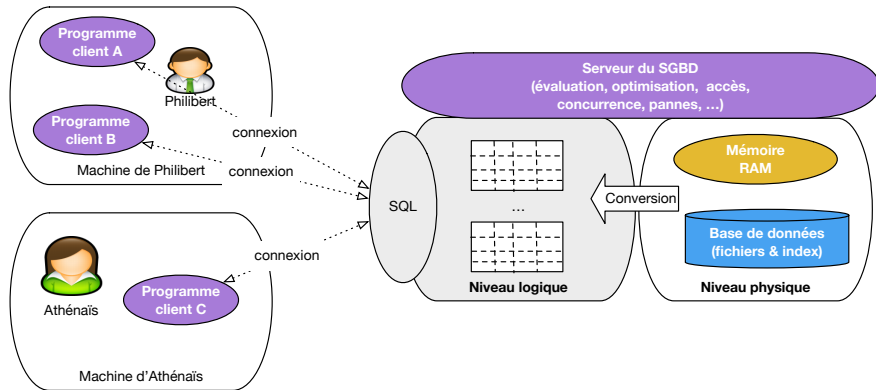


Cours de bases de données,
aspects systèmes,
<http://sys.bdpedia.fr>

Rappels

Vue d'ensemble d'un système relationnel



Retenir : indépendance entre le niveau logique et le niveau physique. Condition nécessaire de l'administration des SGBD.

Fonctionnalités d'un SGBD

Stockage : les données sont stockées de manière ordonnée sur des supports persistants

Indexation : des structures de données permettent des accès très rapides à la base

Interrogation et mise à jour : les requêtes de haut niveau d'abstraction (SQL) sont transcrites en algorithmes efficaces

Partage / Concurrency d'accès : de très nombreuses applications peuvent accéder simultanément à une même base

Sécurité : contrôle des accès et gestion des pannes logicielles ou matérielles

Rappels sur le modèle relationnel (par l'exemple)

Les données sont structurées en **tables** (relations) et **lignes** (nuplets). La ligne est l'unité d'information.

idArtiste	nom	prénom	annéeNaiss
100	Eastwood	Clint	1930
101	Hackman	Gene	1930
102	Scott	Tony	1930
103	Smith	Will	1968

Les données sont structurées et contraintes par un **schéma**.

```
create table Artiste (idArtiste INTEGER NOT NULL,  
                      nom VARCHAR (30) NOT NULL,  
                      prénom VARCHAR (30) NOT NULL,  
                      annéeNaissance INTEGER,  
                      PRIMARY KEY (idArtiste));
```

Clé primaire, clé étrangère

Les tables sont liées par des valeurs partagées entre clé primaire et clé étrangère.

idFilm	titre	année	genre	idMES	codePays
20	Impitoyable	1992	Western	100	USA
21	Ennemi d'état	1998	Action	102	USA

```
create table Film (idFilm integer NOT NULL,  
                  titre    varchar (80) NOT NULL,  
                  année    integer NOT NULL,  
                  genre    varchar (20) NOT NULL,  
                  idRéalisateur integer,  
                  codePays  varchar (4),  
                  primary key (idFilm),  
                  foreign key (idRéalisateur)  
                      references Artiste(idArtiste),  
                  foreign key (codePays)  
                      references Pays(code));
```

Les langages

Le modèle relationnel s'appuie sur deux langages très différents mais **équivalents** en pouvoir expressif.

- SQL a pour fondement la logique des prédicats.

C'est un langage **déclaratif** qui indique ce que l'on veut obtenir sans spécifier **comment** on calcule le résultat.

- L'**algèbre relationnel** est un langage fonctionnel composé d'opérateurs.
L'algèbre permet de construire des **plans d'exécutions** indiquant comment on évalue une requête.

Evaluation et optimisation

Le SGBD transforme une requête au niveau logique (SQL) en un programme d'évaluation algébrique au niveau physique.

Le langage SQL : supposé connu !

Une sélection

```
select titre  
from Film  
where année = 2016
```

Une jointure

```
select titre, prénom, nom  
from Film as f, Artiste as a  
where f.idRéalisateur = a.idArtiste  
and année = 2016
```

La méthode de calcul n'est jamais spécifiée.

Algèbre relationnelle : : supposé connue également

Opérateurs qui prennent une ou deux tables en entrée et produisent une table en sortie.

- Sélection(σ) : Restriction sur valeur des tuples
- Projection(π) : Suppression de colonnes (allège le résultat)
- Jointure($\bowtie$) : Fusionne deux ensembles sur valeur
- Différence($-$) : Sous ensemble non présent dans le second
- Union($\cup$) : Union des deux ensembles

Les opérateurs sont **composables** pour construire des **expressions**

$$\pi_{titre}(\sigma_{année=2016}(Film))$$

$$\pi_{titre,prénom,nom}(\sigma_{année=2016}(Film) \bowtie_{idRéalisateur=idArtiste} Artiste)$$

À retenir

Suivre et comprendre un cours sur les aspects systèmes des SGBDs relationnels suppose des connaissances préalables

- Sur l'architecture des systèmes relationnels, et notamment **l'indépendance entre niveau physique et niveau logique**
- Sur les principes de conception des schémas relationnels et leurs contraintes
- Le langage SQL
- Et l'algèbre relationnelle

À réviser d'urgence si vous avez des doutes ou des lacunes.

Cours de bases de données,
aspects systèmes,
<http://sys.bdpedia.fr>

Supports de stockage

Techniques de stockage

Contenu de ce cours :

- Où sont les données ? Mémoires RAM, mémoire persistante.
- Les performances
- Disques, SSD

Vocabulaire

Une "donnée" désigne une unité physique d'information : pour nous, une ligne d'une table, codée sous forme d'un enregistrement (*record*).

Ces diapositives correspondent au support en ligne disponible sur le site <http://sys.bdpedia.fr/stock.html#s1-supports-de-stockage>

Problématique du stockage

Un SGBD gère deux types de mémoire.

- La mémoire centrale, **volatile**, rapide mais risque de perte.
- La mémoire secondaire, **persistante**, stable mais lente.

Les données sont **toujours** en mémoire secondaire dans des fichiers, et **partiellement** copiées en mémoire centrale.

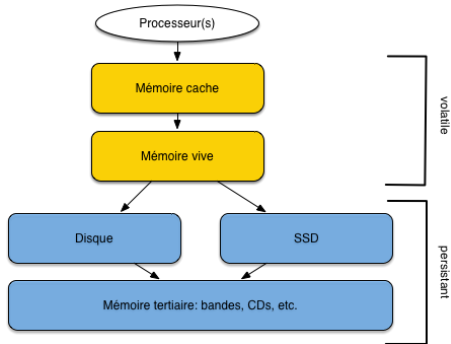
Une donnée, pour être traitée, doit impérativement être en mémoire centrale.

Comment gérer au mieux ces deux ressources ?

Les mémoires d'un ordinateur

Les mémoires dans un ordinateur forment une hiérarchie.

Constat : plus une mémoire est rapide, plus elle est chère , et moins elle est volumineuse.



Performances des mémoires

Critères essentiels :

- **Temps d'accès** : connaissant l'adresse d'une donnée, quel est le temps nécessaire pour aller la chercher à l'emplacement mémoire indiqué par cette adresse ?
Critère essentiel pour les opérations dites **d'accès direct**.
- **Débit** : volume de données lues par unité de temps dans le meilleur des cas.
Critère essentiel pour les opérations dites de **lecture séquentielle**.

Constat : tout accès aux données s'effectue avec l'une de ces deux opérations.

Quelques ordres de grandeur

Rappel : 1 Go = 1 000 Mo et 1 To = 1 000 Go.

Mémoire	Taille	Temps d'accès	Débit
cache	Qq Mo	$\approx 10^{-8}$ (10 nanosec.)	10+ Go/s
Principale (RAM)	Qq Go	$\approx 10^{-8} - 10^{-7}$ (10-100 nanosec.)	Qq Go/s
Disque	Qq To	$\approx 10^{-2}$ (10 millisc.)	100 Mo/s
SSD	Qq To	$\approx 10^{-4}$ (0,1 millisc.)	1+ Go/s

Un accès **direct** disque est (environ) un million de fois plus coûteux qu'en mémoire principale !

Le débit d'un disque est 20, 30, 40 fois plus lent que le débit RAM.

Un SSD est 10 à 100 fois plus performant qu'un disque magnétique.

Importance pour les bases de données

Un SGBD doit ranger sur disque les données ;

- parce qu'elles sont trop volumineuses (de moins en moins vrai) ;
- parce qu'elles sont *persistantes* (et doivent survivre à un arrêt du système).

Le SGBD doit **toujours** amener les données en mémoire pour les traiter.

Si possible, les **données utiles** devraient résider le plus possible en mémoire.

Principe général : la capacité à placer les données **utiles** au plus près du processeur (donc en RAM si possible) est un facteur essentiel de performance.

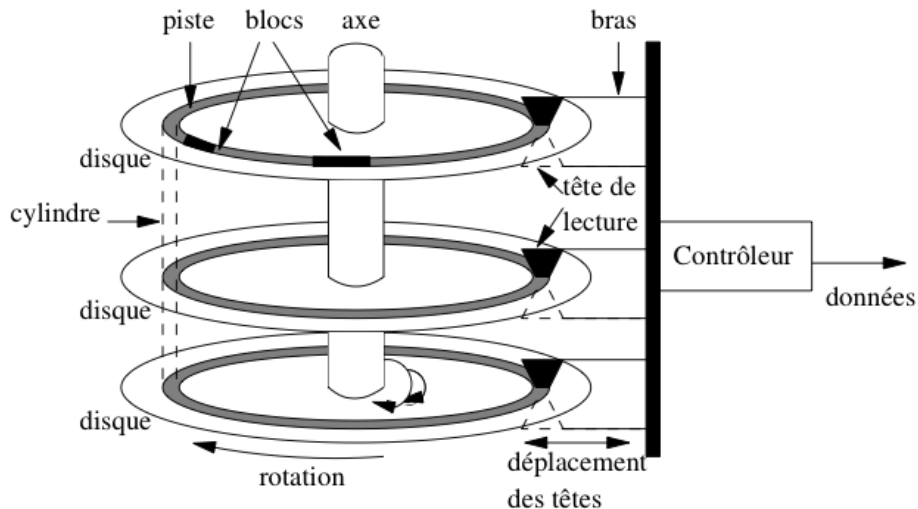
Organisation d'un disque

Disque : surface magnétique, stockant des 0 ou des 1, divisé en secteurs.

Dispositif : les surfaces sont entraînées dans un mouvement de rotation ; les têtes de lecture se déplacent dans un plan fixe.

- Le **bloc** est un ensemble de secteurs, sa taille est en général un multiple de 512 ;
- La **piste** est l'ensemble des blocs d'une surface lus au cours d'une rotation ;
- le **cylindre** est un ensemble de pistes situées sous les têtes de lecture.

Structure d'un disque



Disque = accès semi-direct = latence

Adresse = numéro du disque ; de la piste où se trouve le bloc ; du numéro du bloc sur la piste.

- **Délai de positionnement** pour placer la tête sur la bonne piste :
- **Délai de latence** pour attendre que le bloc passe sous la tête de lecture ;
- **Temps de transfert** pour attendre que le (ou les) bloc(s) soient lus et transférés.

La latence d'un disque : on ne peut pas lire une donnée avant qu'elle passe sous une tête de lecture : la **latence** (qq ms) est un facteur pénalisant.

Granularité d'entrée/sortie : le bloc

Granularité : on lit toujours au moins un bloc, même si on ne veut qu'un octet ; le **bloc** est l'**unité d'entrée/sortie** !

⇒ le remplissage des blocs, si possible avec des données “proches” de la données requise initialement, est essentiel.

Localité : la latence en pratique dépend de la proximité des adresses successives demandées par l'application.

Principe : si deux données sont **proches** du point de vue applicatif, alors elles doivent être proches sur le disque (idéalement, dans le même bloc).

Résumé

La gestion des disques n'est pas directement sous le contrôle de l'administrateur.

Retenir :

- Accès (aléatoire) au disque : très lent par rapport à un accès en RAM (qq. ms).
- Débit du disque : faible (100 MO/s au mieux)
- SSD : 100 fois plus rapide qu'un disque (latence), 10 fois plus en débit

Règle générale : définir des données compactes (types) et stockées contiguement.

Organisation des données, les algorithmes d'accès, la concurrence : tout est conçu pour minimiser les accès aux disques, et surtout les accès **aléatoires**.

Cours de bases de données,
aspects systèmes,
<http://sys.bdpedia.fr>

Gestion des mémoires

Gestion des mémoires

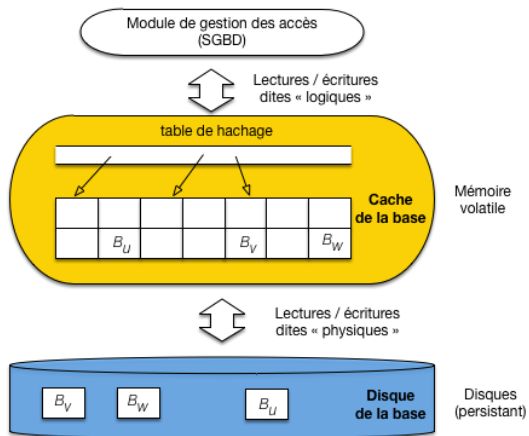
Contenu de ce cours :

- Mémoires gérées par un SGBD : mémoire cache, disque.
- La gestion des lectures.
- La gestion des écritures.
- Quelques applications du principe de localité.

Ces diapositives correspondent au support en ligne disponible sur le site <http://sys.bdpedia.fr/stock.html#s2-gestion-des-memoires>

Le cache et le disque

Paramétrage des SGBD : partie de la mémoire centrale qui sert de *cache*.



Opération de lecture

Toute opération de lecture contient l'adresse du bloc à lire.

- **Si la donnée fait partie d'un bloc dans le cache**, le SGBD prend le bloc, accède à la donnée dans le bloc, et la retourne ;
- **Sinon** il faut d'abord lire un bloc du disque, et le placer dans le cache, pour se ramener au cas précédent.

Hypothèse de localité : le SGBD garde en mémoire les blocs après utilisation, afin d'exploiter à la fois

- la **localité spatiale** : les autres données du bloc.
- la **localité temporelle** : il est probable qu'on va relire la donnée dans peu de temps.

Lectures logiques, lectures physiques : le *Hit ratio*

Le paramètre qui mesure l'efficacité d'une mémoire cache est le *hit ratio* :

$$\textit{hit ratio} = \frac{\text{nb de lectures logiques} - \text{nb lectures physiques}}{\text{nb de lectures logiques}}$$

- Si toutes les lectures logiques (demande de bloc) aboutissent à une lecture physique (accès au disque), le *hit ratio* est 0.
- Aucune lecture physique (tout est en mémoire) : le *hit ratio* est de 1.

En pratique. Un bon *Hit Ratio* est supérieur à 80%, voire 90% : presque toutes les lectures se font en mémoire !

Comment avoir un bon *Hit ratio* ?

Evident :

- il faut allouer le plus de mémoire possible au SGBD.
- il faut limiter la taille de la base (attention aux longs textes, aux images, aux données binaires...)

Mais surtout : il faut que les données utiles soient en mémoire centrale

Intuition : certaines parties de la base sont **beaucoup** plus lues que d'autres : il faut qu'elles tiennent en mémoire.

Stratégie de remplacement

Que faire quand la mémoire est pleine ?

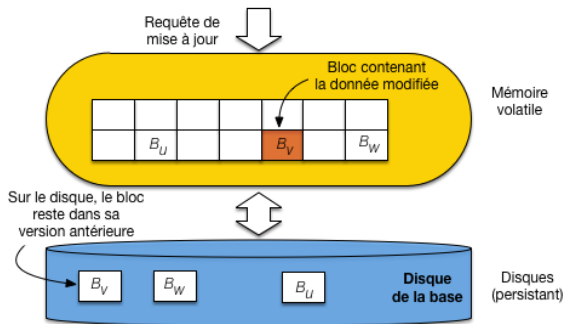
On applique le principe de localité : l'algorithme le plus courant est dit *Least Recently Used* (LRU).

- La "victime" est le bloc dont la dernière date de lecture logique est la plus ancienne.
- Ce bloc est alors soustrait de la mémoire centrale
- Le nouveau bloc vient le remplacer.

Conséquence : le contenu du cache est une image fidèle de l'activité **récente** sur la base de données.

Opération de mise à jour

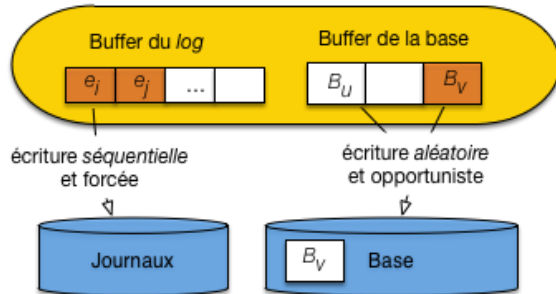
Approche naïve : on trouve le bloc (comme pour une lecture), on modifie la donnée.



Faut-il écrire le bloc ou non ? Deux mauvais choix (pourquoi ?)

La bonne méthode ; le fichier journal

On gère la base (accès aléatoire) **et** un fichier séquentiel, le **journal** (log).



Problème résolu : données sur disque (sûres) et écritures séquentielles.

Le principe de localité et ses conséquences

Principe de localité : l'ensemble des données utilisées par une application pendant une période donnée forme souvent un groupe bien identifié.

- **Localité spatiale** : si une donnée d est utilisée, les données proches de d ont de fortes chances de l'être également.
- **Localité temporelle** : quand une application accède à une donnée d , il y a de fortes chances qu'elle y accède à nouveau peu de temps après.
- **Localité de référence** : si une donnée d_1 référence une donnée d_2 , l'accès à d_1 entraîne souvent l'accès à d_2 .

Les systèmes exploitent ce principe en déplaçant dans la hiérarchie des mémoires des groupes de données.

Résumé

Un critère essentiel de la performance est la taille de la mémoire cache.

Pour vérifier que le système est bien paramétré : on calcule le *Hit Ratio*. Il doit être supérieur à 80%

Sinon, les pistes :

- Vérifier qu'on ne lit que des données utiles.
- Regarder si on peut limiter la taille des données (ex : partitionner la table en mettant les colonnes volumineuses à part).
- Ajouter de la mémoire...

Cours de bases de données,
aspects systèmes,
<http://sys.bdpedia.fr>

Organisation des fichiers

Enregistrements, blocs et fichiers

Contenu de ce cours : la représentation **physique** d'une base de données (par le SGBD).

- les enregistrements et leur adressage
- le stockage des enregistrements dans des blocs
- la séquence des blocs formant un fichier.

Une application n'accède **jamais** directement à la représentation physique.

Ces diapositives correspondent au support en ligne disponible sur le site <http://sys.bdpedia.fr/stfichiers.html>

Enregistrements

Un enregistrement = une suite de *champs* stockant les valeurs des attributs.

Type	Taille en octets
INTEGER	4
FLOAT	4
DOUBLE PRECISION	8
DECIMAL (M, D)	$M, (D+2 \text{ si } M < D)$
CHAR(M)	M
VARCHAR(M)	$L+1$ avec $L \leq M$

Certains champs ont une taille **variable** ; d'autres peuvent être à NULL : pas de valeur.

Tailles variables et valeurs NULL

Si tous les champs avaient de taille fixe et ont une valeur : pas de problème.

En pratique : certains champs ont une taille variable ou sont à NULL.

- pour les champs de taille variable : on précède la valeur par la taille exacte ;
- pour les valeurs NULL : on peut indiquer une taille 0 (Oracle) ; on peut créer un « masque » de bits.

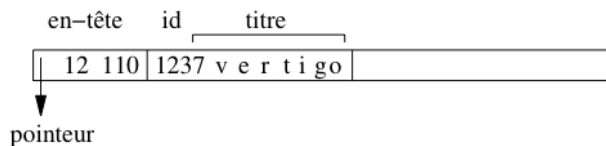
La norme n'impose pas de choix, mais dans tous les cas il faut gérer une **information complémentaire** sur les enregistrements, et la placer dans un **entête**.

En-tête d'enregistrement

Les informations complémentaires sont stockées dans l'en-tête d'un enregistrement.

Exemple :

- table *Film* (id INT, titre VARCHAR(50), année INT)
- Enregistrement (123, 'Vertigo', NULL)

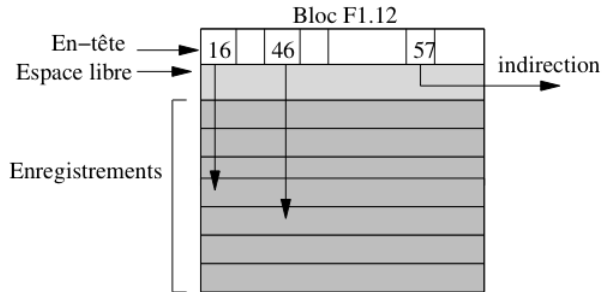


Le contenu de l'enregistrement se décrypte au moment de la lecture.

Blocs

- on veut qu'un enregistrement soit stocké de manière **stable** pour avoir une adresse **fixe** ; **MAIS** :
- on doit envisager le cas où la taille d'un enregistrement varie ;
- on doit envisager une méthode pour gérer un déplacement si nécessaire.

Une solution (Oracle) : chaque bloc gère les adresses internes de ses enregistrements.



Adressage des enregistrements

Avec le schéma précédent, l'**adresse** d'un enregistrement est constituée

- D'une adresse **physique**, celle du bloc. Ex : F1.12
- D'une adresse **logique**, interne au bloc. Ex : 16

L'adresse est **fixe** et utilisée pour les **accès direct** (indexation).

Avec cette solution :

- un enregistrement peut changer de taille, avec réorganisation interne ;
- Plus de place dans le bloc ? On peut déplacer l'enregistrement.

Réorganisation du stockage

Un enregistrement s'agrandit, mais qu'il reste de la place dans le bloc :

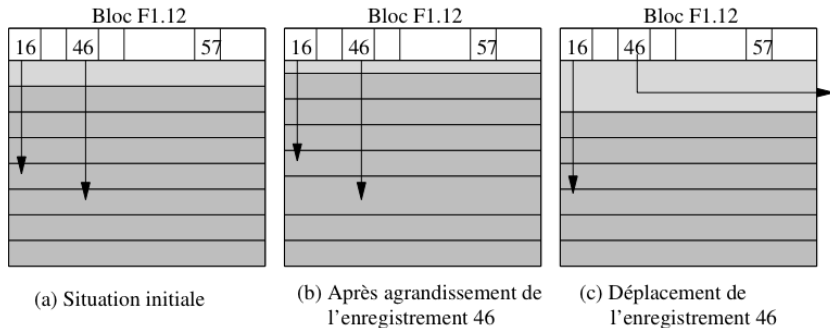
- On modifie l'organisation interne ; la table **locale** d'adressage est modifiée

Un enregistrement s'agrandit, plus de place dans le bloc :

- on déplace l'enregistrement et on crée un **chaînage dans l'en-tête du bloc**.

La création de chaînage pénalise les performances $\Rightarrow$ si possible
laisser de l'espace libre dans un bloc.

Exemple d'évolution avec chaînage



On agrandit deux fois successivement l'enregistrement F1.12.46.

Fichier = séquence de blocs

Un fichier est simplement une séquence de blocs contenant les données d'un même objet (table, index).

La qualité d'un fichier est conditionnée par :

- la bonne utilisation de l'espace (idéalement tous les blocs sont pleins) ;
- le stockage le plus contigu possible (même piste, même cylindre, etc).
- peu ou pas de chaînage

Si les blocs sont très dispersés sur le disque, on aboutit à une **fragmentation** très pénalisante.

Opérations sur un fichier

Voir le support. Essentiellement :

- En l'absence d'index, le seul moyen de rechercher un enregistrement est de parcourir **séquentiellement** le fichier.
- Si le fichier est trié il est possible d'effectuer des recherches par **dichotomie**.

En général, les fichiers gérés par les SGBD sont de simple fichiers séquentiels, sans ordre.

Résumé

La gestion des fichiers est entièrement à la charge du SGBD.

Retenir :

- Les enregistrements sont stockés dans des blocs.
- Le contenu des blocs s'adapte au changement de taille d'un enregistrement après insertion.
- **Il faut un dispositif stable d'adressage de chaque enregistrement**
- Les fichiers sont des séquences de blocs, les moins fragmentés possibles.

L'adressage stable est essentiel pour permettre l'indexation et l'accès direct aux enregistrements.